# COMP1511 26T3 · 第 5 周（W5）知识点整理

编写日期：2026-10-15 · 资料来源：仅限课程公开网站 https://cgi.cse.unsw.edu.au/~cs1511/26T3/（中文概括，非原文转载；技术术语与代码保留英文）

**本周主题一览**：综合示例——2D array of structs + enums（Mud and Bones 小游戏，为 Assignment 1 铺路）→ Assignment 1 style 提示 → 字符串复习（Kahoot、word_guess）→ **command line arguments**（`argc` / `argv`、`atoi`）→ **pointers**（地址、`&`、`*`、通过指针修改变量、stack、函数中的指针、指向 struct 的指针与 `->`、数组与指针）。
W6 为 Flexibility week，没有讲座和 tut-lab（有 help sessions 与 revision sessions），需完成 Ass1 与 Lab 5。Lab 5 截止 **W7 周一 18:00**。

## 1. 综合示例：2D array of structs（Mud and Bones）

讲座用一个小游戏把前几周内容串起来：8×8 地图是 `struct location` 的二维数组，每格有地面类型（enum：草地 / 泥地）和物品（enum：空 / 骨头）。玩家（小狗）用 w/a/s/d 移动：捡到骨头计数并从地图移除；踩泥计数，并把泥带到下一格；找齐骨头获胜，踩泥 3 次失败，Ctrl+D 结束。实现按 stage 递进：① 读入并校验起始位置（不合法时设为 (0, 0)）；② 用循环处理移动直到 Ctrl+D；③ 胜负判定与泥的扩散。

```
#define ROWS 6
#define COLS 6

enum terrain {SAND, WATER};
enum loot {NOTHING, COIN};

struct cell {
    enum terrain ground;
    enum loot item;
};

void init_grid(struct cell grid[ROWS][COLS]) {
    for (int r = 0; r < ROWS; r++) {
        for (int c = 0; c < COLS; c++) {
            grid[r][c].ground = SAND;
            grid[r][c].item = NOTHING;
        }
    }
}

// 检查坐标是否在地图内
int in_bounds(int r, int c) {
    return r >= 0 && r < ROWS && c >= 0 && c < COLS;
}
```

```
#define UP 'w'
#define DOWN 's'

char move;
while (scanf(" %c", &move) == 1) {     // Ctrl+D 时 scanf 不再返回 1
    int new_r = r;
    if (move == UP) {
        new_r--;
    } else if (move == DOWN) {
        new_r++;
    }
    if (in_bounds(new_r, c)) {         // 先检查边界再访问数组
        r = new_r;
    }
    if (grid[r][c].item == COIN) {
        coins++;
        grid[r][c].item = NOTHING;
    }
}
```

### 1.1 Assignment 1 的 style 提示（讲座）

- 按 Style Guide 写；尤其注意：拆分成函数；用 `#define` 替代 magic number（包括 `'w'` 这类命令字符）；写注释；控制行长度。
- 获得反馈的途径：`1511 style`、对照 Style Guide、请 tutor 或 help session 的 tutor 看代码。

## 2. Command line arguments

- 此前输入都在程序**运行后**读取（`scanf`、`fgets`）；命令行参数让你在**启动程序时**就传入输入：`./prog 12 apple`。
- 需要把 main 改为 `int main(int argc, char *argv[])`：
  - `argc`：参数个数，**包括程序名本身**；
  - `argv`：字符串数组，`argv[0]` 永远是程序名；用引号括起来的 `"two words"` 算一个参数。
- 参数都是字符串；需要数字时用 `<stdlib.h>` 的 `atoi()` 转换为 int。

```c
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    if (argc < 2) {                          // 没给参数时打印用法
        printf("Usage: %s <numbers...>\n", argv[0]);
        return 1;
    }
    int product = 1;
    for (int i = 1; i < argc; i++) {         // 从 1 开始，跳过程序名
        product *= atoi(argv[i]);
    }
    printf("Product: %d\n", product);
    return 0;
}
// $ ./mult 2 3 4   → argc 为 4，输出 Product: 24
```

## 3. Pointers（指针）

### 3.1 内存与地址

- 内存可以看作一个巨大的字节数组；**地址**就是这个数组的"下标"，通常用十六进制（`0x` 前缀）表示，本系统上一个地址占 8 字节。
- 每个变量存放在某个地址上；变量的值改变时地址不变。
- **Address-of operator `&`**：取得变量地址，可用 `printf("%p", &x)` 打印。这正是 `scanf("%d", &x)` 中 `&` 的含义——告诉 scanf 把数据送到哪里。

### 3.2 声明、初始化、解引用

```c
int level = 3;
int *level_ptr;            // 1. 声明：指向 int 的指针
level_ptr = &level;        // 2. 初始化：存入 level 的地址（level_ptr "points to" level）
printf("%d\n", *level_ptr);// 3. dereference：取该地址处的值 → 3
*level_ptr = 10;           // 通过指针间接修改 level → level 变为 10

double *d_ptr;             // 其它类型同理
char *c_ptr;
struct cell *cell_ptr;
```

注意 `*` 的两种含义：声明时表示"这是一个指针"；在表达式中表示"解引用"。

### 3.3 Stack 与为什么需要指针

- **Stack** 存放每次函数调用的数据（局部变量、返回位置等）；函数结束时这些数据被自动移除。
- 普通变量是 pass by value：函数改的是拷贝，函数结束后拷贝就消失了——所以 W3 那种"在函数里修改参数"的写法对调用者无效。
- 解决方法：像 `scanf` 一样**传入地址**，函数参数声明为指针，通过 `*` 修改原变量（传进去的仍是地址的拷贝，但它指向原变量）。

```
void add_bonus(int *score, int bonus) {
    *score = *score + bonus;      // 修改调用者的变量
}

void swap(int *a, int *b) {
    int tmp = *a;
    *a = *b;
    *b = tmp;
}

int main(void) {
    int s = 70;
    add_bonus(&s, 5);              // 传地址 → s 变为 75
    int p = 1, q = 2;
    swap(&p, &q);                  // p = 2, q = 1
    return 0;
}
```

### 3.4 指向 struct 的指针与 ->

```
struct pos {
    int row;
    int col;
};

// 方案一：不用指针，返回修改后的 struct
struct pos step_right(struct pos p) {
    p.col++;
    return p;                      // 调用处写 here = step_right(here);
}

// 方案二：传指针，直接修改原 struct
void step_down(struct pos *p) {
    p->row++;                      // 等价于 (*p).row++，-> 更简洁
}
```

### 3.5 数组与指针

- 把数组传入函数时，实际传入的是**首元素的地址**（不是全部数据的拷贝）——这就是函数能修改数组内容的原因。
- 指向数组中某个元素的指针也可以用下标访问，例如 `char *p = &word[2];` 后，`p[0]` 就是 `word[2]`，`printf("%s", p)` 从该位置开始打印剩余字符串。

## 4. 常见坑（Common pitfalls）

| 坑 | 说明 / 正确做法 |
|---|---|
| 移动/访问 2D 数组前不检查边界 | 越界运行时错误；先判断是否 in bounds |
| 忘了 `argc` 包含程序名 | 真正的参数从 `argv[1]` 开始；访问前检查 `argc` 是否足够 |
| 直接把 `argv[i]` 当数字用 | 它是字符串，要先 `atoi`（需 `#include <stdlib.h>`） |
| 声明指针后未初始化就解引用 | 指向未知位置，出错 |
| 混淆 `ptr`（地址）与 `*ptr`（值） | 赋值 `ptr = &x`；改值 `*ptr = 5` |
| 函数参数是指针，调用时却传了值（漏 `&`） | 类型不匹配 |
| 以为 `swap(int a, int b)` 能交换 | 只交换了拷贝；需用指针 |
| 通过 struct 指针访问成员时写 `*p.row` | 优先级错误；写 `(*p).row` 或 `p->row` |
| 字符串比较仍用 `==`，或读字符串用 `scanf("%s")` | 用 `strcmp` / `fgets`（Lab 5 的 debugging 题与 Word Square 都考这一点） |
| Ass1 中命令字符写成 magic number | 用 `#define` 定义 |

## 5. 本周 Tutorial（Week 05）

1. **2D arrays practice**：完成一个"银河探索"小程序——5×5 的 struct 二维数组，每格存天体类型（enum：恒星、行星、星云、飞船、空）和分数；先循环读入行星与星云（输入 `q` 结束），再读入玩家起始位置，然后读入恒星直到输入结束，最后用提供的函数打印地图。
2. **Arrays and functions**：分组把第 1 部分的程序重构为多个函数。
3. **Strings**：热身——逐字符打印到 `'\0'`、用 `%s` 打印、`fgets` / `fputs`；练习——借助已给出的字符判断/转换辅助函数（是否大小写、是否元音、大小写转换等），分组实现一个字符串处理函数（如统计小写字母个数）。

## 6. 本周 Lab（Lab 05）

截止：**W7 周一 18:00**（中间隔着 flexibility week）。读字符串一律用 `fgets`，不要用 `scanf("%s")`。部分题要求通过 `1511 style`，否则该题最高 75%。

| 难度 | 题目 | 内容概括 | 提交命令 |
| --- | --- | --- | --- |
| ☆ | Check Off | 课内与 tutor 完成 | — |
| ●◌◌ | Debugging - stringcmp（要求 style） | 修复一个"读入两个字符串并判断是否相同"的程序 | `give cs1511 lab05_debug_stringcmp debug_stringcmp.c` |
| ●◌◌ | Word Square（pair） | 用 fgets 读入一个单词，重复打印它 n 次（n 为单词长度） | `give cs1511 lab05_word_square word_square.c` |
| ●◌◌ | Alternating case | 补全函数：把字符串改为首字母小写、之后字母大小写交替 | `give cs1511 lab05_alternating_case alternating_case.c` |
| ●◌◌ | Creating a simple Snake game | 在 2D 地图上实现简化版贪吃蛇：生成蛇与苹果、移动、吃苹果变长（不实现撞到尾巴结束） | `give cs1511 lab05_simple_snake simple_snake.c` |
| ●●◌ | Planter（pair） | 5×5 花园模拟：每轮为一行选择花的种类或不种；花需 3 轮开花，未开时显示剩余轮数，开花后显示首字母；每轮打印花园，种完后继续等待开花 | `give cs1511 lab05_planter planter.c` |
| ●●◌ | Palindrome | 补全 `is_palindrome`：判断输入单词是否回文，是返回 1，否则 0 | `give cs1511 lab05_palindrome palindrome.c` |
| ●●● | Rewrite Letters (Repeated) | Lab 4 Rewrite Letters 的进阶：把替换规则重复应用多次 | `give cs1511 lab05_rewrite_letters_repeated rewrite_letters_repeated.c` |
| ●●● | Largest Z Sum | 补全函数：在方形 2D 数组中找出所有 "Z" 形状里数字和最大的那个 | `give cs1511 lab05_largest_z_sum largest_z_sum.c` |
| ☠ | Implement p1511 | 实现一门新的玩具编程语言（esoteric language）的解释器 | （详见题面） |

### 6.1 Extra Revision 与 Challenges（可选）

- Revision 05：若干 ●●◌ 练习，以及 Perfect Square Numbers、两数组点积、2D 数组最大值、Temperature、2D 数组各列平均值、Weekly Earnings、最快与最慢跑者、Finding Treasure 等。
- Challenges 05（均 ●●●）：Pointer Path、Math with Function Pointers、Conway's Game of Life。

## 7. 本周讲座示例代码（Lecture Code）

| 目录 | 文件 | 演示内容 |
|---|---|---|
| 0.mud_and_bones | mud_and_bones_starter.c、mud_and_bones.c | 2D array of structs + enums 综合游戏 |
| 1.kahoot | q2.c … q9_10.c | 字符串 Kahoot 题目代码 |
| 2.words | word_guess_starter.c、word_guess.c | 字符串综合练习 |
| 3.cml | cml.c、atoi_sum.c、bad_sum.c | 命令行参数与 atoi |
| 4.pointers/0.basics | pointer_intro.c、changing_pointers.c、pointer_exercise.c | 指针基础 |
| 4.pointers/1.functions | pass_by_value_int(_pointer).c、pass_by_value_struct(_pointer).c | 传值 vs 传指针（int 与 struct） |

## 8. Check-off 自测题 推测

官方 check-off 页面（/check-off/05/）需 zID 登录，**具体题目「未发布」（对公众不可见）**。以下为依据本周内容**推测**的自测题，只列问题、不给答案。

1. 【推测】运行 `./prog a "b c" 7` 时 `argc` 是多少？`argv[2]` 是什么？
2. 【推测】为什么命令行参数里的数字需要 `atoi`？
3. 【推测】`&x`、`int *p`、`*p` 分别是什么意思？
4. 【推测】给一段包含两个指针互相赋值的代码，写出它的输出。
5. 【推测】为什么 `swap(int a, int b)` 不能交换 main 中的变量？怎样修改？
6. 【推测】`p->x` 与 `(*p).x` 有什么关系？
7. 【推测】为什么函数可以修改传入的数组，却不能修改传入的 int？
8. 【推测】在 2D 地图上移动角色时，访问数组前要做什么检查？
9. 【推测】如何判断一个字符串是否为回文？
10. 【推测】`scanf(" %c", ...) == 1` 作为循环条件起什么作用？

## 9. W6 与 W7+ 说明

- **W6：Flexibility week。**没有讲座和 tut-lab，但有 help sessions 和 revision sessions（W5 幻灯片称具体安排"稍后公布"）。这一周用来完成 Assignment 1 和 Lab 5。课程网站上没有 W6 的幻灯片目录（返回 404）。
- **W7 及以后：「未发布」。**W7–W10 的 tutorial / lab 页面在网站导航中标注为 "tutors only"；W7、W8 的幻灯片目录存在但尚无 PDF。W5 讲座预告 W7 将讲 heap、`malloc`、dynamic arrays 和 linked lists（W7、W8 由另一位讲师授课）。

## 来源页面（本章引用）

- 课程首页：https://cgi.cse.unsw.edu.au/~cs1511/26T3/
- Week 5 Lecture 1 幻灯片：https://cgi.cse.unsw.edu.au/~cs1511/26T3/slides/week_5/COMP1511_26T3_Week_5_Lecture_1.pdf

- Week 5 Lecture 2 幻灯片：https://cgi.cse.unsw.edu.au/~cs1511/26T3/slides/week_5/COMP1511_26T3_Week_5_Lecture_2.pdf
- Week 5 Lecture Code：https://cgi.cse.unsw.edu.au/~cs1511/26T3/code/week_5/
- Tutorial 05：https://cgi.cse.unsw.edu.au/~cs1511/26T3/tut/05/questions
- Lab 05：https://cgi.cse.unsw.edu.au/~cs1511/26T3/lab/05/questions
- Revision 05：https://cgi.cse.unsw.edu.au/~cs1511/26T3/revision/05/questions
- Challenges 05：https://cgi.cse.unsw.edu.au/~cs1511/26T3/challenges/05/questions
- Check-off 05（需登录，未能访问）：https://cgi.cse.unsw.edu.au/~cs1511/26T3/check-off/05/

说明：本文件为中文概括整理，代码示例均为改写的简短示例；以课程官网为准。